

Exercícios — Aula 14 — BFS em matrizes e uso de filas

Técnicas de Programação - 2026/02

Última atualização: October 5, 2026

Instruções

- Em BFS, marque uma posição quando ela entra na fila.
- Considere apenas movimentos para cima, baixo, esquerda e direita.
- # representa parede e . representa uma posição livre.

Exercício 1 — Simulação da fila na BFS

Considere o labirinto. A origem é S; o destino é D.

```
S . .  
. # D  
. . .
```

Algorithm 1: DistanciasBfs

Result: Calcula distâncias mínimas a partir da origem.

Input : labirinto lab, origem

Output: matriz dist

```
dist ← MatrizPreenchida(-1)  
fila ← FilaVazia()  
dist[origem] ← 0  
Enfileirar(fila, origem)  
while Vazia(fila) = false do  
    atual ← Desenfileirar(fila)  
    foreach vizinho ortogonal de atual do  
        if vizinho é livre and dist[vizinho] = -1 then  
            dist[vizinho] ← dist[atual] + 1  
            Enfileirar(fila, vizinho)  
        end  
    end  
end  
return dist
```

Complete os quatro primeiros passos da execução, usando a ordem cima, baixo, esquerda e direita.

passo	posição removida	posições enfileiradas	fila ao final	novas distâncias
1				
2				
3				
4				
5				
6				

Exercício 2 — Matriz de distâncias

Preencha a matriz com a distância mínima a partir de **S**. Mantenha **#** para paredes e use **-1** para posições livres inalcançáveis.

```
S . . #  
. # . #  
. . . D
```

```
0 _ _ #  
_ # _ #  
_ - - -
```

Em qual camada da BFS o destino é descoberto?

Exercício 3 — Reconstrução de caminho

Uma BFS registrou os predecessores abaixo. Cada seta aponta da posição descoberta para a posição de onde ela veio.

$(2, 3) \leftarrow (2, 2) \leftarrow (1, 2) \leftarrow (0, 2) \leftarrow (0, 1) \leftarrow (0, 0)$

1. Escreva o caminho da origem até o destino na ordem correta.
2. Quantos movimentos ele possui?
3. Que valores devem ser armazenados para a posição $(1, 2)$ nas matrizes de predecessores de linha e coluna?

Exercício 4 — Completar menor distância

Complete o procedimento. Ele deve retornar -1 se o destino não for alcançado.

Algorithm 2: MenorDistancia

Result: Encontra o menor número de movimentos em um labirinto.

Input : lab, origem, destino

Output: int

```
dist[origem]  $\leftarrow$  0
Enfileirar(fila, origem)
while Vazia(fila) = false do
    atual  $\leftarrow$  Desenfileirar(fila)
    if atual = destino then
        | return _____
    end
    foreach vizinho ortogonal de atual do
        | if vizinho é livre and dist[vizinho] =  $-1$  then
        | | dist[vizinho]  $\leftarrow$  _____
        | | _____
        | end
    end
end
end
return _____
```

Por que a atribuição da distância precisa ocorrer antes de enfileirar o vizinho?

Exercício 5 — Por que a primeira chegada é mínima?

Explique, com suas palavras, por que a primeira vez que BFS alcança o destino corresponde ao menor número de movimentos.

Use obrigatoriamente as ideias de:

- fila;
- camadas de distância 0, 1, 2, ...;
- marcação na descoberta;
- todos os movimentos terem o mesmo custo.

Depois, explique por que DFS pode encontrar um caminho válido sem encontrar o menor.

Exercício 6 — Quando BFS simples não basta?

Classifique cada situação.

Situação	BFS simples resolve menor caminho?	Justificativa
cada movimento custa 1		
mover na areia custa 3 e na estrada custa 1		
há teleporte que também custa 1		
movimentos diagonais também custam 1		
queremos apenas saber se existe algum caminho		

Para a segunda situação, explique por que “menor número de movimentos” e “menor custo” podem ser perguntas diferentes.